Patrol Scrap – Sistema de venta por diálogo

Oct 30, 2026 · @Duke

Visión general

El sistema de venta convierte el botín en dinero mediante un diálogo con clientes cuyo límite de precio y paciencia están ocultos al jugador.

Es la fase de tarde de Patrol Scrap. Tras descargar la chatarra en la tienda, llegan clientes que piden piezas. El jugador acepta, rechaza o negocia, y el dinero que gana paga la gasolina del día siguiente y las mejoras del coche.

Cumple el pilar de **Negociación** del GDD: una fase rápida pero tensa, con una «paciencia oculta» en los clientes y tres tipos especiales que el jugador debe aprender a identificar. La mecánica de diálogos se inspira en *Papers, Please*.

Estado actual:

- **Hecho (pasos 1 y 2):** cartera con dinero en pantalla, y los datos y el generador de clientes.
- **Pendiente (pasos 3 a 6):** lógica del regateo, interfaz de diálogo, flujo de la tienda, y gasolina de pago con garaje.

Las piezas, una a una

Seis scripts forman la parte ya construida. Cada uno hace una sola cosa y se comunica con los demás sin depender de ellos.

Script	Qué hace	Concepto clave
`TiendaDestino`	Guarda el **stock**: el botín descargado, por material	Diccionario `material → cantidad`
`Cartera`	Guarda el dinero y lo cobra o paga	Evento `Cambiado`
`DineroUI`	Dibuja el contador, con animación	Escucha el evento de `Cartera`
`TipoCliente`	Define a un tipo de cliente con todos sus números	`ScriptableObject`
`Cliente`	Un cliente concreto en el mostrador, con sus valores ya sorteados	Clase de datos, no `MonoBehaviour`

Script	Qué hace	Concepto clave
`GeneradorClientes`	Elige tipo, pieza y cantidad de cada cliente nuevo	Selección ponderada

TiendaDestino: el stock

Recibe cada pieza que aterriza de la descarga y suma su cantidad al material correspondiente. Expone `Stock` en solo lectura para que nadie lo modifique desde fuera. Es lo que el generador consulta para saber qué puede pedir un cliente.

Cartera: el dinero

Guarda un `int`, no un `float`, porque el dinero no debe tener errores de redondeo. Ofrece cuatro operaciones:

- `Ingresar(n)`: cobra una venta.
- `TryPagar(n)`: paga; devuelve `false` y no resta nada si no hay dinero suficiente.
- `PuedePagar(n)`: pregunta si alcanza, útil para activar o desactivar botones.
- `Establecer(n)`: carga una partida guardada (TDD: sistema de guardado en JSON).

Cada cambio lanza el evento `Cambiado(total, variación)`. `Cartera` no sabe que existe la pantalla.

DineroUI: el contador

Escucha `Cambiado` y anima tres cosas: el número cuenta hasta el valor nuevo con una curva que frena al llegar, el panel late y se tiñe de verde o rojo, y aparece un `+50` flotante que sube y se desvanece. Si el evento no llegara, compara el dinero con el último valor conocido cada frame y anima igualmente.

TipoCliente: los datos de un cliente

Es un asset que se crea en el proyecto, como `MaterialChatarra`. Reúne la identidad (nombre, color), el peso de aparición, los rangos de **límite** y **oferta inicial**, la **paciencia**, la sensibilidad a la codicia, el umbral de ofensa y las frases de diálogo. Equilibrar el juego es cambiar números en el Inspector, sin tocar código.

Cliente: el cliente del mostrador

No es un objeto de la escena, solo datos. Al crearse sortea sus valores dentro de los rangos de su tipo: precio de referencia, límite oculto, oferta inicial y paciencia. Después mantiene lo que cambia durante el regateo: la oferta actual y la paciencia restante.

GeneradorClientes: quién viene

Consulta el stock, elige un material que exista en él, decide la cantidad (de 1 al mínimo entre lo disponible y el máximo por pedido) y escoge el tipo con una ruleta ponderada por peso. Si el stock está vacío devuelve `null`: sin mercancía no viene nadie.

Los cuatro clientes

Hay un cliente normal y los tres especiales del GDD. Cada uno premia una forma distinta de negociar, y el jugador debe aprender a reconocerlos.

Tipo	Peso (aparición)	Límite (× precio ref.)	Oferta inicial (× su límite)	Paciencia	Pérdida por ronda	Se ofende si pides
Normal	45	1,00 – 1,25	0,70 – 0,85	80 – 100	15	nunca
Regateador	25	0,60 – 0,80	0,55 – 0,70	110 – 140	12	nunca
Impaciente	20	1,15 – 1,45	0,80 – 0,90	30 – 45	25	nunca
Rico	10	1,60 – 2,20	0,60 – 0,75	90 – 120	10	+35 % sobre su límite

Los pesos suman 100, así que equivalen a probabilidades: un 45 % de clientes normales, un 25 % de regateadores, un 20 % de impacientes y un 10 % de ricos.

Cómo se reconoce cada uno

El jugador nunca ve el límite ni la paciencia. Tiene que deducirlos de tres pistas: el **color y aspecto** del retrato, el **tono de las frases** y la **primera oferta**.

Tipo	Pista visual y de diálogo	Estrategia correcta
Normal	Sin señal particular	Negociar un poco; aceptar si la oferta es razonable
Regateador	Ropa remendada; abre muy por debajo del precio y se queja	Empujar con paciencia: su límite es bajo, pero aguanta muchas rondas
Impaciente	Mira el reloj, frases cortas y exclamaciones	Aceptar rápido: paga bien, pero se va a la primera ronda o a la segunda
Rico	Anillos y traje caro; abre por encima del precio	Subir con medida: paga mucho, pero se ofende si abusas

Qué aprende el jugador

- **Aceptar a la primera es un mal trato con el Regateador**: abre en torno a 0,44 veces el precio y su límite real es 0,70.
- **Con el Impaciente y el Rico, aceptar es rentable**: abren por encima del precio de referencia. La tentación es pedir más, y ahí se pierde la venta.
- **Cada tipo cambia el ritmo de la fase de tienda**, lo que evita la monotonía de atender siempre al mismo cliente.

Las fórmulas del sistema

Todo parte del precio de referencia y de unos pocos sorteos dentro de los rangos de cada tipo.

1. Precio de referencia, el «precio de catálogo» del pedido:

```
P_{ref} = ValorBase_{material} \times cantidad
```

2. Límite oculto, lo máximo que pagaría, con un factor sorteado entre el mínimo y el máximo de su tipo:

```
L = P_{ref} \times f_{limite}, \qquad f_{limite} \in [f_{min},\, f_{max}]
```

3. Oferta inicial, nunca por encima del límite:

```
O_0 = \min\bigl(L,\; L \times f_{oferta}\bigr)
```

4. Probabilidad de aparición, la ruleta ponderada por peso:

```
P(tipo) = \frac{peso_{tipo}}{\sum peso}
```

Ejemplo completo

Llega un **Rico** que quiere 3 unidades de Cobre, con `ValorBase` 25:

Paso	Cálculo	Resultado
Precio de referencia	25 × 3	75 $
Factor de límite sorteado	entre 1,60 y 2,20	1,84
Límite oculto	75 × 1,84	138 $
Factor de oferta sorteado	entre 0,60 y 0,75	0,66

Paso	Cálculo	Resultado
Oferta inicial	138 × 0,66	91 $
Paciencia	sorteada entre 90 y 120	104

El cliente abre con 91 $, por encima del precio de referencia, pero pagaría hasta 138 $. El jugador no ve esos 138 $: solo puede probar hasta dónde sube sin que se enfade.

Cuántas rondas aguanta cada tipo

Sin contar la penalización por codicia, la paciencia media dividida entre la pérdida por ronda da el número de rondas de margen:

Tipo	Paciencia media	Pérdida por ronda	Rondas aprox.
Normal	90	15	6
Regateador	125	12	10
Impaciente	38	25	1,5
Rico	105	10	10

Por eso el Impaciente exige decidir rápido y el Regateador deja insistir.

Lo que falta por construir

Quedan cuatro pasos, y el siguiente es el más importante: la lógica del regateo. Todo lo que sigue es **diseño previsto**, aún sin programar; las reglas y los valores pueden cambiar al probarlos.

Paso	Qué se construye	Cómo se prueba
3	Lógica del regateo, sin interfaz: aceptar, rechazar y negociar	Por consola
4	Interfaz de diálogo estilo *Papers, Please*, con ratón o joystick derecho	En pantalla
5	Flujo de la tienda: clientes que llegan a por el stock y fin del día	Juego completo
6	Pagar la gasolina del día siguiente (sustituye al llenado automático temporal) y después el garaje	Juego completo

Paso 3: las reglas del regateo previstas

El jugador tiene tres acciones con cada cliente.

- **Aceptar:** vende a la oferta actual del cliente.
- **Rechazar:** el cliente se va y no hay venta.
- **Negociar(precio):** el jugador propone un precio y el cliente reacciona según sus números ocultos.

Al negociar, el cliente resuelve en este orden:

1. Si el precio pedido es igual o menor que su **límite**, acepta y se cierra la venta.
2. Si lo pedido supera su límite en más del **umbral de ofensa**, se marcha de golpe (solo el Rico tiene umbral).
3. Si no, hace una **contraoferta**: sube su oferta una fracción (**cesión por ronda**) de la distancia que le separa de su límite.
4. Pierde paciencia: la **pérdida por ronda** más un extra proporcional a lo codicioso del precio (**sensibilidad a la codicia**).
5. Si la paciencia llega a cero, se va.
6. Si le queda menos de la fracción de **aviso**, suelta una frase de impaciencia: la pista que el jugador puede leer.

La mecánica de «paciencia oculta» del GDD es exactamente esto: el jugador no ve la barra, solo sus efectos.

Paso 6: dinero con consecuencias

Hoy, al reaparecer, el depósito se llena solo. Con la venta, rellenarlo costará dinero y será lo primero que se pague, como pide el GDD; lo que sobre irá a las mejoras del garaje. `TryPagar` ya está preparado para eso.

Decisiones de diseño y su porqué

Cada decisión responde a un objetivo del GDD o a un riesgo del TDD. Conviene saber justificarlas.

Decisión	Por qué
El jugador **no ve** el límite ni la paciencia	Es la «paciencia oculta» del GDD: la tensión viene de adivinar hasta dónde se puede empujar

Decisión	Por qué
Cuatro tipos de cliente con estrategias distintas	Evita la monotonía de atender siempre al mismo cliente y da algo que aprender
Cada tipo es un **asset de datos** (`ScriptableObject`)	Equilibrar el juego es cambiar números en el Inspector, sin recompilar ni tocar código
`Cartera` y `DineroUI` **separadas**	La lógica del dinero no depende de la pantalla; la tienda, el garaje y el guardado reutilizan la misma cartera
Comunicación por **eventos** (`Cambiado`)	Quien escucha se entera sin que `Cartera` lo conozca; añadir un nuevo oyente no obliga a tocarla
`TryPagar` devuelve `bool`	El dinero nunca queda en negativo, y quien llama decide qué hacer si no alcanza
Dinero como `int`	Evita errores de redondeo propios de `float`
Los clientes piden solo lo que hay en **stock**	Hace coherente el diálogo: nunca pide algo que no puedes dar
Frases con **marcadores** (`{pieza}`, `{precio}`)	Una misma frase sirve para cualquier pedido, y el tono diferencia al tipo de cliente
`Cliente` es una **clase de datos**, no un objeto de escena	Es más ligero y fácil de probar; la interfaz solo lo muestra

Por qué se construye por pasos

Se separó el sistema en datos, lógica y pantalla para poder probar cada capa por separado. La lógica del regateo se probará por consola antes de dibujar la interfaz: si falla, se sabe que el fallo no está en la pantalla.

Relación con el TDD

- **Guardado en JSON:** `Cartera.Establecer(n)` ya existe para cargar el dinero guardado.
- **Guía de estilo:** clases en PascalCase, variables privadas con guion bajo, `[SerializeField]` en lugar de campos públicos.
- **Rendimiento:** la fase de tienda no es crítica (GDD), y la interfaz solo se redibuja cuando algo cambia.

Guion de exposición

Una demo de unos cinco minutos que va del botín al cliente, usando los botones de prueba del Inspector para no depender del azar.

Antes de empezar

- Juego en marcha con la pestaña **Game** visible.
- La tienda con el botín del último viaje, o stock de prueba cargado.
- El Inspector de `Economia` abierto, para lanzar los botones de prueba.

Pasos de la demo

1. **Contexto (30 s).** «Patrol Scrap tiene dos fases: por la mañana se conduce y se recoge chatarra; por la tarde se vende en la tienda. Este sistema es la segunda fase.»
2. **El botín llega a la tienda (45 s).** Vuelve al aro de la base. Los cubos vuelan hasta la tienda y sube el stock. «Lo recogido es el stock del que compran los clientes.»
3. **El dinero (60 s).** En `Cartera`, lanza *TEST: ingresar 50*, luego *pagar 30*. Señala el pulso verde o rojo y el número flotante. Lanza *pagar 100000*: la consola dice `False` y el dinero no cambia. «`TryPagar` protege el dinero: nunca queda en negativo.»
4. **Un cliente (60 s).** En `GeneradorClientes`, lanza *generar un cliente* y lee la línea de consola. «Cada cliente tiene un precio de referencia, un límite que no ve el jugador y una paciencia oculta.»
5. **Los cuatro tipos (60 s).** Lanza *generar 200 y ver el reparto*. Enseña el porcentaje de cada tipo y su límite medio. «El Regateador abre muy por debajo del precio, el Rico por encima, y el Impaciente aguanta una o dos rondas.»
6. **Cómo se cambia el equilibrio (30 s).** Abre un asset `Cliente_Rico` y cambia el rango de límite. «No hay que tocar código: se equilibra desde el Inspector.»
7. **Lo que sigue (30 s).** «El siguiente paso es la lógica del regateo y la interfaz de diálogo estilo *Papers, Please*.»

Si algo falla durante la demo

Fallo	Qué hacer
No sale ningún cliente	La tienda está vacía: lanza *llenar stock de prueba*
No se ve la animación del dinero	Comprueba que estás en la pestaña **Game**, no en **Scene**
Los porcentajes no cuadran con la tabla	Es normal con solo 200 clientes; varían unos puntos

Fallo	Qué hacer
Un botón de prueba no hace nada	Solo funcionan con el juego en marcha

Preguntas probables

¿Por qué el jugador no ve la paciencia del cliente? Porque la tensión del regateo nace de la incertidumbre. Si hubiera una barra, bastaría con vigilarla; así tiene que leer las frases y la primera oferta para deducir cuánto puede empujar.

¿Por qué usar `ScriptableObject` para los clientes? Son datos que viven en el proyecto, no en una escena. Se crean varios tipos con el mismo script, se editan en el Inspector y se pueden equilibrar sin tocar código ni recompilar.

¿Qué es un evento y por qué se usa? Es un aviso que una clase lanza cuando algo cambia, sin saber quién escucha. `Cartera` avisa de que el dinero cambió y `DineroUI` reacciona. Así se pueden añadir más oyentes sin modificar la cartera.

¿Por qué el dinero es un `int`? Porque un `float` acumula errores de redondeo. Con monedas enteras, las cuentas son siempre exactas.

¿Cómo se decide qué tipo de cliente aparece? Con una ruleta ponderada. Cada tipo tiene un peso, se suma el total, se tira un número al azar y se recorre la lista restando pesos hasta que el número cae en uno.

¿Por qué un cliente solo pide lo que hay en stock? Para que el diálogo sea coherente: nunca pide algo que el jugador no puede dar. Si la tienda está vacía, no viene nadie.

¿Qué impide que el jugador pierda dinero por error? `TryPagar` solo resta si hay saldo suficiente. Si no lo hay, devuelve `false` y no cambia nada.

¿Cómo se evita que el Rico o el Regateador sean siempre la mejor opción? Sus números se ajustan desde el asset: límite, oferta inicial y paciencia. Se comprueba con el test de reparto, que muestra el límite medio y la oferta media de cada tipo.

¿Qué falta por hacer? La lógica del regateo (aceptar, rechazar, negociar), la interfaz de diálogo, el flujo de la tienda y el pago de la gasolina con el garaje. Está planificado en cuatro pasos.

¿Cómo se relaciona con el guardado del TDD? `Cartera` tiene `Establecer(n)` para cargar el dinero desde el JSON, y el stock de la tienda se guardará con el mismo sistema.

Glosario

Término	Significado
Stock	Botín ya descargado en la tienda, por material
Precio de referencia	`ValorBase` del material × cantidad pedida: el «precio de catálogo»
Límite	Lo máximo que pagaría un cliente. Oculto para el jugador
Oferta inicial	Lo que propone el cliente al llegar, como fracción de su límite
Paciencia	Puntos ocultos que se gastan en cada ronda; a cero, el cliente se va
Cesión por ronda	Fracción de la distancia hasta su límite que sube su oferta cada ronda
Sensibilidad a la codicia	Paciencia extra que pierde por cada 10 % que pides sobre su límite
Umbral de ofensa	Porcentaje sobre el límite a partir del cual el cliente se va de golpe
Peso	Probabilidad relativa de que aparezca un tipo de cliente
Ronda	Una acción de negociar: el jugador propone un precio y el cliente responde
`ScriptableObject`	Asset de datos de Unity que vive en el proyecto, no en una escena
Evento	Aviso que una clase lanza al cambiar algo, sin conocer a quien escucha
`MonoBehaviour`	Clase base de los scripts que se añaden a objetos de la escena
Selección ponderada	Elegir al azar donde unas opciones pesan más que otras
Marcador	Texto como `{precio}` que se sustituye por el valor real al mostrar la frase
Paciencia oculta	Mecánica del GDD: el jugador solo percibe los efectos de la paciencia del cliente